Divine Word College of Legazpi
School of Engineering and Computer Sciences
Department of Computer Sciences and Information Technology
AY 2026-2027, 1st Semester

Integrated Student Organization Activity Approval and Fund Release Platform

Prefinal Project Proposal
ITPC 313 – System Integration and Architecture

Team name: FAQsilica

Role	Name
Project Lead / Architect	Jessa Angela Sabueto
Integration Developer	Alex Mendoza Jr.
Data and Security Lead	Joey Manlapas
QA and Documentation Lead	Faith Red
Instructor	Neal Errol Bulawam
Date submitted	November 5, 2026

2. Executive summary

This project connects the registry, approval, finance, and student information systems of a school so that a student organization activity is approved and funded in one traceable flow, without re-encoding data by hand.

Student organizations are expected to plan activities, get them approved, and receive funds before the activity takes place. In many schools this runs on paper forms, spreadsheets, and personal follow-ups across several offices. The same organization and officer details are written again at each step, the organization's remaining budget is checked by hand, and officers cannot see where their request is stuck.

The proposed integration layer connects four systems: a Student Organization Registry System (SORS), an Activity Approval Workflow System (AAWS), a Finance and Budget System (FBS), and a Student Information System (SIS), with a Notification Service for status messages. An API gateway checks every request's token and role. Organization and officer data are held once in SORS and shared through REST APIs. When the last approver accepts an activity, AAWS publishes an ActivityApproved event through a RabbitMQ message broker, and FBS automatically creates the disbursement request. When the cashier releases the funds, FBS publishes a FundReleased event so that AAWS and the officers see the status without any re-entry.

The expected benefit is a shorter approval-to-release cycle, no duplicate encoding of organization data, budget checks before approvers spend time on a request, and a complete audit trail of who approved what and when. The system will be built with open-source tools, run through Docker Compose, use synthetic data only, and be judged against eight measurable quality attribute scenarios (Section 7.2), for example that 95% of disbursement requests are created within 3 seconds at 50 concurrent users and that no approval event is lost or duplicated.

3. Background and problem statement

Data about organizations, officers, approvals, and budgets sits in separate offices that do not exchange it electronically, so people re-encode the same details by hand and chase signatures to learn the status of a request.

3.1 The organization

The case organization is a higher education institution, Divine Word College of Legazpi. It has accredited student organizations; the Student Affairs and Services Office (SASO), which accredits organizations and approves their activities; college deans and organization advisers who review proposals; a Finance Office that manages the student activity fund; and a Cashier who releases cash or checks.

3.2 Current process

The process below is the typical paper-based flow. Each step is handled in a different office and, most often, in a different file or logbook.

Table 1. Current process and where data is siloed or re-encoded

Step	Office	Medium	Silo or re-encoding point	Cost
1. Prepare activity proposal and budget	Student organization	Paper or word-processed form	Organization name, officers, adviser, and activity details are typed from scratch each time	Duplicate work, typing errors
2. Adviser and dean endorsement	Adviser, Dean	Physical signature	No shared record; status known only by asking	Waiting time, lost forms
3. SASO review and approval	SASO	Logbook or spreadsheet	Officer eligibility and accreditation checked by hand against separate lists	Delays, inconsistent checks
4. Budget availability check	Finance Office	Ledger or spreadsheet	Approved amount is checked against the organization’s fund only after approval	Approved requests later rejected for lack of funds
5. Voucher and fund release	Finance, Cashier	Paper voucher	Approved details are re-encoded into the voucher	Transcription errors, slow release
6. Liquidation	Student organization, Finance	Receipts and forms	Liquidation is matched to the original request by hand	Unclear closure status, weak audit trail

3.3 Problem statement

Because the offices do not exchange data electronically, people re-encode the same details by hand, check budgets late, and chase signatures to learn the status of a request. Hohpe and Woolf (2003) describe this situation as the typical starting point for integration work: several applications that each hold part of a business process and must exchange data reliably to support it. The result is slow approvals, preventable errors, duplicate work, and weak traceability of fund release decisions.

The problem this project addresses is therefore: **how can the registry, approval workflow, finance, and student information systems be integrated so that an activity is approved and**

funded from a single, traceable flow of data, without re-encoding and without losing messages when one system is unavailable?

4. Objectives

General objective: To design and prototype an integration architecture that connects the student organization registry, activity approval workflow, finance and budget, and student information systems so that activity approval and fund release run as one automated, secure, and traceable process.

Table 2. Specific objectives and how they will be measured

#	Specific objective	How it will be measured
1	Remove manual re-encoding of organization and officer data by reading it from SORS through an API	Count of fields typed more than once in the end-to-end test flow: target 0
2	Reject requests that exceed the organization's available budget before they reach approvers	100% of test requests above the available balance are rejected at submission; 0 false approvals
3	Create the disbursement request in FBS automatically when the final approval is given	95% of ActivityApproved events produce a disbursement request within 3 seconds at 50 concurrent users
4	Keep the process correct when a system or the broker is down or a message repeats	0 lost and 0 duplicate disbursement requests in the failure and replay tests (Section 14)
5	Record every status change and access decision for audit	100% of state transitions in the test flow are logged with actor, time, and old and new state

5. Scope and limitations

The prototype covers the approval-to-release flow end to end and simulates the systems the team cannot access.

Table 3. Scope of the project

In scope	Out of scope	Mocked (simulated)
Integration of SORS, AAWS, and FBS through REST APIs behind an API gateway	Real bank transfers or online payment of funds	Student Information System (SIS) as a stub with synthetic student records
Event-driven flow from final approval to disbursement request and fund release using a message broker	Real student, officer, or financial data	Notification Service (messages written to a log or test mailbox)
Canonical data model with JSON Schema and field mapping	Full accounting features (general ledger, taxes, procurement)	Cashier confirmation of release (a test endpoint that marks funds as released)
Role-based access control with token authentication	Mobile application and polished user interface	School single sign-on / identity provider (a simple token issuer is used)
Audit logging of status changes	Production hosting and operations	
Docker Compose deployment, OpenAPI contracts, and automated integration, load, and failure tests	Accreditation of new organizations and annual renewal workflow	

5.1 Limitations

- The prototype models one approval chain (adviser, SASO, dean); other schools may use different steps.
- SORS, AAWS, and FBS are simplified services that stand in for the school's real systems; they demonstrate integration, not full feature parity.
- Load and failure tests run on the team's own machines, so results show relative behavior against the stated targets, not production capacity.
- All data are synthetic or anonymized, as required for this course.

6. Systems to be integrated

Five systems exchange data in the proposed design; the technology choices are justified in the architecture decision records (Section 12).

Table 4. Systems to be integrated

System	Owner	Purpose	Technology stack	Data it holds	Interface it exposes
Student Organization Registry System (SORS)	Student Affairs and Services Office (SASO)	Master record of accredited organizations, officers, and advisers	Node.js (Express), PostgreSQL	Organization ID, name, accreditation status, officer list with roles, adviser	REST: organizations, officers, eligibility check
Activity Approval Workflow System (AAWS)	SASO with college deans	Receives activity proposals and routes them through approval stages	Node.js (Express), PostgreSQL	Proposal, budget request, approval steps and decisions, status history	REST: proposals, decisions, status; publishes ActivityApproved
Finance and Budget System (FBS)	Finance Office	Tracks organization fund balances, disbursement requests, release, and liquidation	Node.js (Express), PostgreSQL	Organization fund balance, earmarks, disbursement vouchers, liquidation status	REST: balance, disbursements, release; consumes ActivityApproved; publishes FundReleased
Student Information System (SIS), mocked	Registrar	Confirms that a student is currently enrolled and in good standing	Stub service (JSON, in-memory)	Synthetic student ID, enrollment status, program	REST: student status lookup
Notification Service, mocked	IT Services Office	Sends status messages to officers, advisers, and approvers	Stub consumer writing to log or test mailbox	Message templates, recipients (synthetic)	Consumes status events

The integration layer itself (API gateway, message broker, and shared canonical schemas) is the team's own contribution and is designed in Sections 8 and 9.

7. Requirements

The system has nine functional requirements, eight measurable quality scenarios, and four constraints.

7.1 Functional requirements

Table 5. Functional requirements as integration flows

ID	Integration flow or user story	Systems involved
FR-1	As an organization officer, I submit an activity proposal and the system fills in my organization and officer details from the registry so I do not type them again.	AAWS, SORS
FR-2	As an approver, I only see proposals from organizations whose accreditation is active and whose submitting officer is eligible.	AAWS, SORS, SIS
FR-3	As an officer, when I submit a budget, the system checks it against the organization's available fund and rejects it immediately if it is higher.	AAWS, FBS
FR-4	As an adviser, SASO reviewer, or dean, I approve or return a proposal in order, and each decision is recorded with my name and time.	AAWS
FR-5	When the final approver accepts a proposal, AAWS publishes ActivityApproved and FBS creates a disbursement request and earmarks the amount.	AAWS, FBS (via broker)
FR-6	When the cashier releases the funds, FBS publishes FundReleased and AAWS updates the activity status.	FBS, AAWS (via broker)
FR-7	As an officer, I receive a notification at every status change and can view the current status of my proposal.	AAWS, FBS, Notification Service
FR-8	As an officer, I submit liquidation after the activity, and the linked proposal is marked as liquidated or pending.	FBS, AAWS
FR-9	As an administrator, I assign roles (officer, adviser, SASO, dean, finance, cashier, admin) and the APIs allow each role only its own actions.	All systems, gateway

7.2 Quality attribute scenarios

Each scenario names a source, a stimulus, and a measurable response, following the scenario form described by Bass et al. (2021).

Table 6. Quality attribute scenarios

ID	Quality attribute	Scenario	Measure (target)
QA1	Performance	The final approver accepts a proposal while 50 users are active; AAWS publishes ActivityApproved and FBS creates the disbursement request.	95% of disbursement requests are created within 3 seconds
QA2	Performance	An officer submits a proposal and the budget check is made against FBS while 50 users are active.	95% of submissions return an accept or reject result within 2 seconds
QA3	Reliability (availability)	FBS is down when a proposal receives final approval.	0 events lost; 100% of queued events processed within 60 seconds after FBS is restored
QA4	Reliability (idempotency)	The same ActivityApproved message is delivered twice.	Exactly one disbursement request per activity in 1,000 replayed messages

ID	Quality attribute	Scenario	Measure (target)
QA5	Security	A caller sends a request with no token, an expired token, or a role that is not allowed for the action.	100% of 100 test requests rejected with 401 or 403 and logged
QA6	Auditability	An auditor asks for the history of one approved activity.	100% of state changes show actor, time, old and new state; history returned within 5 seconds
QA7	Modifiability	The mocked SIS is replaced by another system that implements the same interface.	Only the SIS adapter changes; 0 changes to AAWS workflow code; done in 1 working day or less
QA8	Data privacy	AAWS sends officer data to FBS for a disbursement request.	100% of payloads contain only fields defined in the canonical schema; no extra personal data

7.3 Constraints

Table 7. Constraints

Constraint	Description
Budget	No paid licenses or cloud services. Open-source tools and free tiers only; team members shoulder no hosting cost.
Hardware	Team laptops. All services, the broker, and databases run on one machine through Docker Compose; load tests are limited to what a single laptop can generate.
Time	Proposal parts 1–7 and repository in Prefinal Week 1; design and full proposal in Week 2–3; build in the Final.
Legal	The Data Privacy Act of 2012 (Republic Act No. 10173) governs personal data of students and officers (National Privacy Commission, n.d.). Only synthetic or anonymized data are used, and the design applies data minimization and access control.

8. Proposed architecture

The architecture puts one API gateway in front of three services for immediate questions and connects the services through a message broker for approvals, fund release, and notifications.

8.1 Architectural style and integration topology

We chose an **API gateway combined with event-driven messaging** (ADR-001). The gateway is a single entry point for users and applies token and role checks. Behind it, REST calls answer questions that need an immediate reply, such as the budget check. The broker acts as a hub for events: AAWS and FBS publish facts such as ActivityApproved and FundReleased, and each interested system consumes them from its own queue (Hohpe & Woolf, 2003; Richards & Ford, 2020).

Table 8. Comparison of integration styles and topologies

Option	Advantages	Disadvantages	Chosen?
Point-to-point REST between systems	Simple to start; no extra component	Caller fails when the other system is down; every pair needs its own contract and mapping; no central security	No
Hub-and-spoke with an enterprise service bus	Central routing and transformation; many ready-made adapters	Heavy to install and learn; the bus becomes a bottleneck; too large for a laptop-based project	No
API gateway with REST only	One entry point; central token and role checks; easy to test	Approval-to-release depends on FBS being up at that moment; no queuing during an outage	No
Event-driven with a broker only	Systems are decoupled; queued events survive an outage	Users have no simple way to ask for an immediate answer such as the budget check; security is repeated in each service	No
API gateway plus event-driven messaging	REST for immediate answers (QA2); queued events for outages and repeats (QA3, QA4); one security entry point (QA5)	Two kinds of interface to document and test; status can lag by a few seconds	Yes

8.2 C4 diagrams

The C4 diagrams show the system at two levels of detail (Brown, n.d.). Figure 1 shows who uses the platform and which outside systems it connects to; Figure 2 opens the platform into its containers.

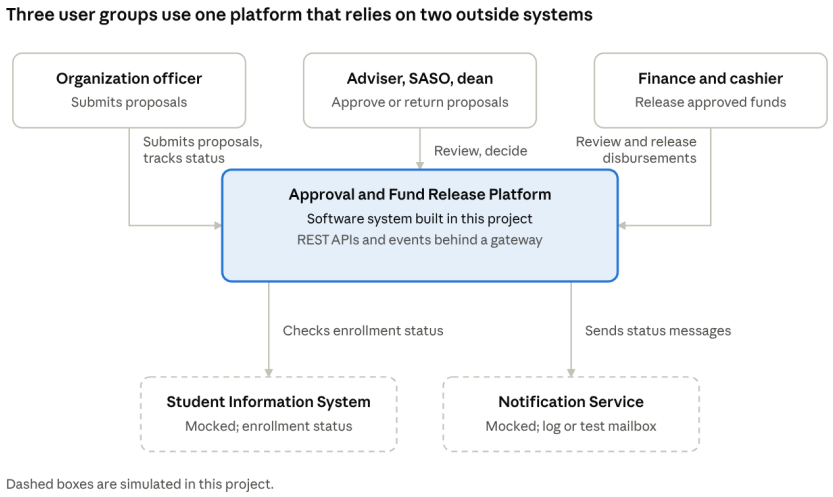


Figure 1. C4 Context diagram of the Integrated Student Organization Activity Approval and Fund Release Platform.

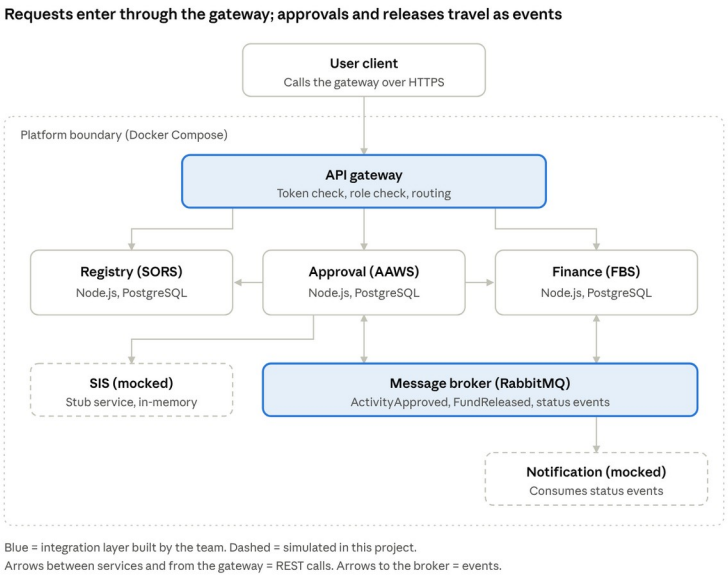


Figure 2. C4 Container diagram of the platform. Each service keeps its own PostgreSQL database; the broker carries the four events in Table 10.

8.3 UML deployment diagram

Figure 3 shows how the containers run on a single laptop through Docker Compose.

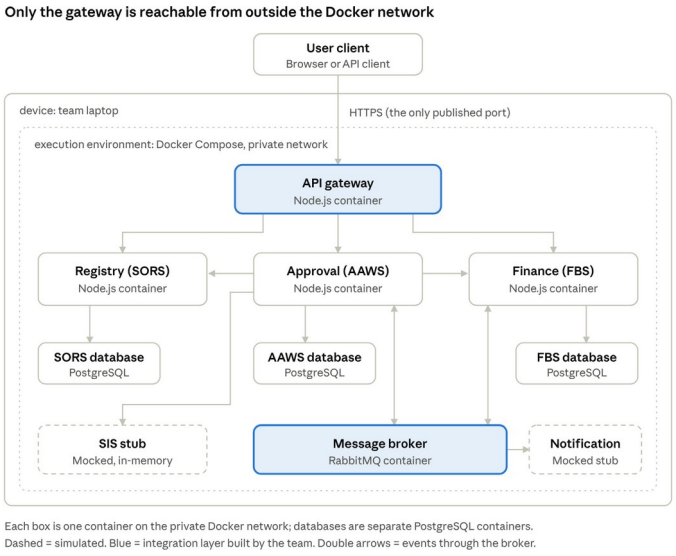


Figure 3. UML deployment diagram of the platform on one team laptop.

9. Integration design

Systems call each other through REST for questions that need an immediate answer, and exchange events for facts that other systems must react to later.

9.1 API contracts

All paths sit behind the API gateway under the base path `/api/v1`. Every call carries a bearer token (Section 10). The draft OpenAPI 3.1 file is `docs/openapi.yaml` in the team repository (Miller et al., 2021).

Table 9. API contracts

Method	Path	Provider	Consumer	Purpose
POST	/auth/token	Gateway token issuer (mocked identity provider)	All clients	Issue a short-lived JWT with the caller's role
GET	/organizations/{orgId}	SORS	AAWS	Get organization name, adviser, and accreditation status
GET	/organizations/{orgId}/ officers/{studentId}/ eligibility	SORS	AAWS	Check that the student is an active officer of the organization
GET	/students/{studentId}/status	SIS (mocked)	AAWS	Check enrollment and good standing
POST	/proposals	AAWS	Officer client	Submit an activity proposal with its budget request
GET	/proposals/{proposalId}	AAWS	Officer and approver clients	Read the current status of a proposal
POST	/proposals/{proposalId}/ decisions	AAWS	Adviser, SASO, dean clients	Approve or return a proposal at the current stage
GET	/proposals/{proposalId}/ history	AAWS	Auditor client	Read every state change with actor and time
GET	/funds/{orgId}/balance	FBS	AAWS	Get the organization's available balance for the budget check
GET	/disbursements/ {disbursementId}	FBS	Officer and finance clients	Read the disbursement request and its status
POST	/disbursements/ {disbursementId}/release	FBS	Cashier client (mocked)	Confirm that funds were released
POST	/liquidations	FBS	Officer client	Submit liquidation for a released activity

9.2 Message and event catalogue

The broker is RabbitMQ with one topic exchange, `campus.events`. Each consumer has its own durable queue, and each queue has a dead-letter queue (`<queue>.dlq`) for messages that fail repeatedly. Every event carries a unique `eventId` so consumers can ignore repeats (Hohpe & Woolf, 2003).

Table 10. Event catalogue

Event or message	Producer	Consumer(s)	Payload (main fields)	Broker queue or topic
ActivityApproved	AAWS	FBS, Notification	eventId, proposalId, organizationId, approvedAmount, approvedAt, approvedBy	Routing key activity.approved; queue fbs.activity-approved
FundReleased	FBS	AAWS, Notification	eventId, disbursementId, proposalId, organizationId, amount, releasedAt, releasedBy	Routing key fund.released; queue aaws.fund-released
LiquidationSubmitted	FBS	AAWS, Notification	eventId, liquidationId, proposalId, status, submittedAt	Routing key liquidation.submitted; queue aaws.liquidation
ProposalStatusChanged	AAWS	Notification	eventId, proposalId, organizationId, oldStatus, newStatus, changedAt, actorId	Routing key proposal.status-changed; queue notif.status

9.3 Canonical data model and mapping

The canonical model is the shared vocabulary that every API and event uses, so that each system translates only once, between its own fields and the canonical ones. It has four entities: **Organization** (organizationId, name, accreditationStatus, adviserName), **Officer** (studentId, organizationId, role), **ActivityProposal** (proposalId, organizationId, title, activityDate, requestedAmount, status) and **Disbursement** (disbursementId, proposalId, amount, status). Money is an integer in centavos and all timestamps are ISO 8601 in UTC.

The JSON Schema below defines the ActivityApproved event (Wright et al., 2022). The schemas for the other events and entities are kept in `docs/schemas/` in the repository.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://faqsilica.example/schemas/activity-approved.json",
  "title": "ActivityApproved",
  "type": "object",
  "required": ["eventId", "proposalId", "organizationId", "approvedAmount", "approvedAt",
"approvedBy"],
  "additionalProperties": false,
  "properties": {
    "eventId": { "type": "string", "format": "uuid" },
    "proposalId": { "type": "string", "format": "uuid" },
    "organizationId": { "type": "string", "format": "uuid" },
    "approvedAmount": { "type": "integer", "minimum": 1, "description": "PHP centavos" },
    "approvedAt": { "type": "string", "format": "date-time" },
    "approvedBy": { "type": "string", "description": "User ID of the final approver" }
  }
}
```

Table 11. Canonical field mapping

Canonical field	System A field	System B field	Transformation
organizationId	SORS organizations.org_id	FBS fund_accounts.org_ref	None; UUID string copied as is
accreditationStatus	SORS organizations.accreditation (ACTIVE, LAPSED)	AAWS eligibility check result (true, false)	ACTIVE maps to true; any other value maps to false
studentId	SORS officers.student_no	SIS students.student_id	Trim spaces and convert to uppercase

Canonical field	System A field	System B field	Transformation
approvedAmount	AAWS proposals.approved_budget (decimal PHP)	FBS disbursements.amount_centavos (integer)	Multiply by 100 and round to an integer
status	AAWS proposals.status_code (for example SUBMITTED, ADVISER_OK)	Notification message text	Map status codes to canonical status names, then to message templates
approvedAt	AAWS proposals.approved_at (local time)	FBS disbursements.received_at (UTC)	Convert local time to ISO 8601 UTC

10. Security and data privacy plan

Every request is authenticated at the gateway, every action is checked against the caller's role, and personal data is limited to what each step needs.

Authentication and authorization: Users obtain a JSON Web Token (JWT) from the token endpoint (a mocked identity provider) and send it as a bearer token (Jones et al., 2015). The token is signed, expires after 15 minutes, and carries the user ID, the role (officer, adviser, SASO, dean, finance, cashier, admin), and the organization ID for officers. The gateway checks the signature and expiry. Each service then checks the role and, for officers, the organization ID against a role-permission table, so an officer can only read and change proposals of their own organization. Service-to-service calls use the OAuth 2.0 client credentials grant with one scope per service (Hardt, 2012).

Transport security: Clients reach the gateway over HTTPS using TLS 1.3 (Rescorla, 2018). Services, databases, and the broker sit on a private Docker network and publish no ports except the gateway. The broker accepts only TLS connections (AMQPS) with a separate user and queue permissions for each service. The prototype uses self-signed certificates; secrets are kept in environment files that are not committed to the repository.

Table 12. Threat model using the OWASP API Security Top 10 (OWASP Foundation, 2023)

OWASP API Security Top 10 risk	Where it applies in our system	Control
API1 Broken object level authorization	GET /proposals/{proposalId}, GET /disbursements/{disbursementId}: an officer could request another organization's record	Compare the organization ID in the token with the record's organization ID; return 403 on mismatch
API2 Broken authentication	POST /auth/token and every bearer token	Short-lived signed tokens; reject missing, expired, or tampered tokens at the gateway; limit failed sign-in attempts

OWASP API Security Top 10 risk	Where it applies in our system	Control
API3 Broken object property level authorization	POST /proposals and decisions: a caller could set fields such as status or approvedAmount	Accept only the fields in the OpenAPI schema; set status and approvedAmount on the server
API4 Unrestricted resource consumption	Proposal submission and balance lookups under load	Rate limit per user at the gateway; cap page size and request body size; timeouts on service calls
API5 Broken function level authorization	Decision, release, and role-assignment endpoints	Role-permission table: only the current stage's approver can decide; only the cashier can release; only admin assigns roles
API6 Unrestricted access to sensitive business flows	Repeated proposal submissions to lock up the organization's fund	One open proposal per activity; the budget earmark is created only after final approval; alert on repeated rejected submissions
API7 Server side request forgery	Services that call other services by URL (for example AAWS calling SORS, FBS, and SIS)	Service URLs come from configuration only, never from request data; outbound calls limited to the Docker network
API8 Security misconfiguration	Gateway, broker, and containers	No default passwords; no published ports except the gateway; TLS enforced; error messages hide stack traces
API9 Improper inventory management	Old or undocumented endpoints and test endpoints such as cashier release	Only paths in the OpenAPI file are routed; mocked endpoints are disabled outside the test profile; versioned base path /api/v1
API10 Unsafe consumption of APIs	AAWS and FBS trusting data from SORS, SIS, and the broker	Validate every response and every event against its JSON Schema; reject unknown fields; ignore events with an already-seen eventId

The design follows the principles of transparency, legitimate purpose, and proportionality in the Data Privacy Act (National Privacy Commission, n.d.). The inventory below lists the personal data that moves between systems. The prototype uses synthetic data only.

Table 13. Personal data inventory (RA 10173)

Personal data item	From system	To system	Purpose	RA 10173 basis	Protection
Officer full name and student ID	SORS	AAWS	Identify the submitting officer on a proposal	Legitimate interests of the school (Sec. 12(f)); officer consent given at accreditation (Sec. 12(a))	Events carry the student ID only; name is read on demand; TLS; role check
Officer role and organization	SORS	AAWS	Check eligibility to submit	Legitimate interests of the school (Sec. 12(f))	Eligibility endpoint returns true or false, not the full officer list
Enrollment status	SIS	AAWS	Confirm the officer is enrolled and in good standing	Legitimate interests of the school (Sec. 12(f))	Only the status value is returned, not grades or program details; not stored after the check
Officer contact email	SORS	Notification Service	Send status messages	Consent of the officer (Sec. 12(a))	Not included in events; Notification Service reads it from SORS when sending
Approver user ID and decision time	AAWS	Audit log, FBS	Accountability for approval and release decisions	Legitimate interests of the school (Sec. 12(f))	Append-only log; read access for auditor and admin roles only

11. Reliability plan

No approval and no fund release may be lost or done twice, even when a system or the broker is down. Specific techniques such as retries, outbox, and circuit breakers will be chosen in Final Week 2; the table states what should happen.

Table 14. Failure handling

Failure	What should happen
One system is down	The caller gets a clear "temporarily unavailable" answer for questions that need an immediate reply, such as the budget check, and the proposal stays in draft so the officer can try again. Events for the down system wait in its queue and are processed in order when it returns, within 60 seconds (QA3). The approval decision itself is never lost.
The broker is down	AAWS and FBS still save their own data and mark the event as "not yet published". When the broker returns, they publish the pending events. Users see the status "approved, waiting for release processing" instead of an error.
The same message arrives twice	The consumer checks the eventId, finds it was already handled, and ignores it. Exactly one disbursement request exists per activity (QA4).
A message cannot be processed	After a limited number of tries, the message moves to the dead-letter queue and the Data and Security Lead is notified to check it. It is not deleted.
A message arrives with unexpected fields	The consumer rejects it against the JSON Schema and logs the reason; no data is saved.

12. Architecture Decision Records

Four decisions shape the design. Each follows the format of Nygard (2011).

ADR-001: Combine an API gateway with event-driven messaging

Status: Accepted | **Date:** 2026-11-05 | **Deciders:** Jessa Angela Sabueto, Alex Mendoza Jr., Joey Manlapas, Faith Red

Context: Questions such as the budget check need an immediate answer (QA2), while the move from approval to disbursement must survive a system outage (QA3). Constraints: one laptop, no paid tools, four team members.

Options considered:

1. Point-to-point REST only – simple, but a down system blocks the caller and every pair needs its own contract.
2. Enterprise service bus – central routing and transformation, but heavy to run and learn within the time limit.
3. API gateway plus message broker (event-driven) – REST for immediate questions, events for follow-up work.

Decision: We chose option 3 because it meets QA2 with REST and QA3 and QA4 with queued events, and both parts run in Docker Compose.

Consequences: Easier: systems can be down independently, and one gateway enforces security (QA5). Harder: eventual consistency, so the status can lag by seconds. Risk accepted: two kinds of interfaces to document and test.

ADR-002: Use RabbitMQ as the message broker

Status: Accepted | **Date:** 2026-11-05 | **Deciders:** Jessa Angela Sabueto, Alex Mendoza Jr., Joey Manlapas, Faith Red

Context: The flow needs durable queues, per-consumer delivery, and dead-letter handling for four events with low volume (tens of messages per minute).

Options considered:

1. RabbitMQ – durable queues, topic routing, dead-letter queues, simple to run in one container; less suited to replaying long histories.
2. Apache Kafka – strong for high-volume streams and replay, but heavier to operate and more than this volume needs.
3. Database table as a queue – no new component, but polling, locking, and no routing features.

Decision: We chose RabbitMQ because it provides the queue, routing, and dead-letter features needed for QA3 and QA4 at the lowest set-up cost on a laptop.

Consequences: Easier: per-service queues and failure handling. Harder: no long-term event replay, so the audit trail is kept in each service's database. Risk accepted: the broker is a single point of failure in the prototype (see Section 11).

ADR-003: Use a canonical data model with JSON Schema validation

Status: Accepted | **Date:** 2026-11-05 | **Deciders:** Jessa Angela Sabueto, Alex Mendoza Jr., Joey Manlapas, Faith Red

Context: SORS, AAWS, FBS, and SIS name and format the same facts differently (for example decimal pesos versus centavos). QA7 requires that replacing a system changes only its adapter, and QA8 requires that payloads hold only defined fields.

Options considered:

1. Direct mapping between each pair of systems – no shared model, but the number of mappings grows with every new system.
2. Canonical model – each system maps once to the shared model; one extra translation step.

Decision: We chose a canonical model, with every API and event validated against a JSON Schema, because it limits changes to one adapter (QA7) and blocks extra personal data (QA8).

Consequences: Easier: replacing the mocked SIS, adding a new consumer. Harder: schema changes need coordination and versioning. Risk accepted: a poorly designed canonical model could become a bottleneck for change.

ADR-004: Use JWT bearer tokens with role checks in every service

Status: Accepted | **Date:** 2026-11-05 | **Deciders:** Jessa Angela Sabueto, Alex Mendoza Jr., Joey Manlapas, Faith Red

Context: Seven roles use the APIs, and officers must only reach their own organization's records (QA5, OWASP API1 and API5).

Options considered:

1. Server-side sessions – simple for browsers, but each service needs shared session storage.
2. Static API keys – easy, but no expiry and no per-user roles.
3. JWT bearer tokens issued by a token endpoint – signed, expiring, carry the role and organization.

Decision: We chose JWT bearer tokens because services can verify them without a shared session store and they carry the role and organization needed for object-level checks.

Consequences: Easier: stateless services, simple tests with prepared tokens. Harder: a token stays valid until it expires, so lifetimes are kept to 15 minutes. Risk accepted: the mocked identity provider is not a real school sign-on.

13. Technology stack and tools

All tools are free and open source and run in Docker Compose on a team laptop. Exact versions are pinned in the repository's lock files and may move to a newer patch release during the build.

Table 15. Technology stack and tools

Layer	Tool	Version	Why
Language / framework	Node.js with Express	Node.js 24 LTS, Express 5	One language for all services keeps a four-person team productive; large library support for REST and AMQP
API gateway	Express gateway with http-proxy-middleware, jsonwebtoken, express-rate-limit	Latest stable at build start	Meets token check, routing, and rate limit needs (ADR-001, ADR-004) without a paid product
Broker	RabbitMQ with the management plugin	4.x	Durable queues, topic routing, and dead-letter queues (ADR-002)
Database	PostgreSQL, one database per service	17	Reliable transactions for balances and audit history; free
Schema and contract tools	OpenAPI 3.1 file, JSON Schema 2020-12, Ajv validator	Ajv 8	One contract per API and event, checked automatically (ADR-003)
Containers	Docker Compose	Compose v2	One command starts every service, the broker, and databases on a laptop
Testing and load testing	Jest and Supertest for integration tests; k6 for load tests	Jest 30, k6 1.x	Automates the QA scenarios in 7.2 and gives percentile timings
Source control and diagrams	GitHub; Structurizr or draw.io for C4 and UML diagrams	Current	Shared repository with the instructor as a member; diagrams stored as files

14. Testing plan

Every quality attribute scenario in Section 7.2 has at least one test with a pass criterion taken from its measure. Tests run on a Docker Compose test profile with synthetic data.

Table 16. Testing plan

Test	What it checks	Tool	Pass criterion
Integration flow test	FR-1 to FR-9 end to end: submit, budget check, three approvals, ActivityApproved, disbursement request, FundReleased, status update	Jest and Supertest	All steps pass; the proposal ends in “released”; 0 fields typed more than once (objective 1)
Budget check test	Requests above and below the organization’s balance (FR-3)	Jest and Supertest	100% of requests above the balance rejected at submission; 0 false approvals (objective 2)
Load test	QA scenarios in 7.2: QA1 approval-to-disbursement time and QA2 budget-check time with 50 virtual users for 5 minutes	k6	95% of disbursement requests created within 3 seconds; 95% of budget checks answered within 2 seconds
Failure test	QA3: stop FBS, approve proposals, restart FBS; also stop the broker during an approval	Docker Compose commands and Jest	0 events lost; 100% of queued events processed within 60 seconds after recovery
Duplicate message test	QA4: publish the same ActivityApproved message repeatedly	Jest with a test publisher	Exactly one disbursement request per activity in 1,000 replayed messages
Security test	QA5: no token, expired token, wrong role, and another organization’s record (OWASP API1, API2, API5)	Jest and Supertest	100% of 100 test requests rejected with 401 or 403 and logged
Audit test	QA6: read the history of an approved activity	Jest	Every state change shows actor, time, old and new state; history returned within 5 seconds

Test	What it checks	Tool	Pass criterion
Contract and schema test	QA7 and QA8: payloads and responses against OpenAPI and JSON Schema; swap the SIS stub for a second stub	Ajv, Jest	100% of payloads contain only schema-defined fields; only the SIS adapter changes

15. Work plan

The design and the full proposal are finished by Prefinal Week 2 and the slides are rehearsed in Prefinal Week 3; every task has one owner. The Final-week plan will be written once the Final requirements are known.

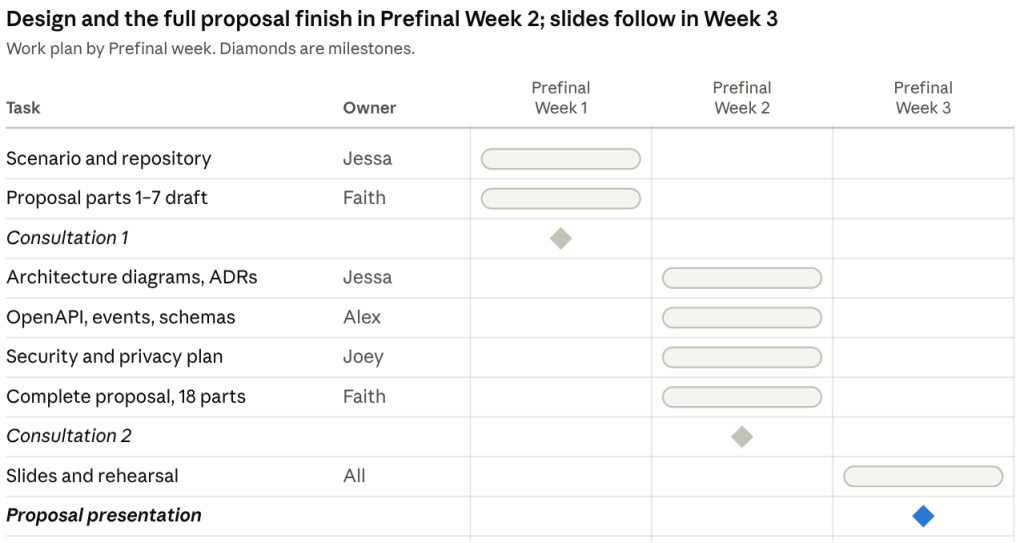


Figure 4. Gantt chart of the work plan, one row per task and one column per Prefinal week (Weeks 1 to 3), with owners and milestones.

R = does the work, A = approves, C = consulted.

Table 17. Responsibility matrix

Task	Jessa Angela Sabueto	Alex Mendoza Jr.	Joey Manlapas	Faith Red
Architecture diagrams and ADRs	R	C	C	A
REST API and broker flows	A	R	C	C
Security and privacy	A	C	R	C
Tests and documentation	A	C	C	R

16. Risks and mitigation

The biggest risks are time, scope, and late discovery of security or performance problems; each has an owner and a first response.

Table 18. Risks and mitigation

#	Risk	Likelihood (Low/Med/ High)	Impact (Low/Med /High)	Response	Owner
1	Docker Compose and broker set-up take longer than planned	Med	High	Build the Compose skeleton with the broker and one test event in the first build week	Alex Mendoza Jr.
2	Scope grows beyond Table 3 (extra roles, user interface features)	Med	Med	Keep the scope table as the baseline; discuss any addition at the next consultation	Jessa Angela Sabueto
3	Load test results vary between laptops	High	Med	Use the same machine and seed data, run each test three times, report the median, and state the limitation	Faith Red
4	A security flaw such as broken object level authorization is found late	Med	High	Write the security tests for each endpoint before its code; review the OWASP table every week	Joey Manlapas
5	A change to an event or API schema breaks a consumer	Med	Med	Version schemas, keep them in one folder, and run contract tests on every change	Alex Mendoza Jr.
6	A member is unavailable or workload is uneven	Low	High	Name an approver and a consulted member for every task in the responsibility matrix; hold short weekly check-ins	Jessa Angela Sabueto
7	Real personal data is used by mistake	Low	High	Generate all data with a seed script, never commit environment files, and review test data before demos	Joey Manlapas

17. References

Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice* (4th ed.). Addison-Wesley.

Brown, S. (n.d.). *The C4 model for visualising software architecture*. Retrieved November 5, 2026, from https://c4model.com/

Hardt, D. (Ed.). (2012). *The OAuth 2.0 authorization framework* (RFC 6749). Internet Engineering Task Force. https://www.rfc-editor.org/info/rfc6749

Hohpe, G., & Woolf, B. (2003). *Enterprise integration patterns: Designing, building, and deploying messaging solutions*. Addison-Wesley.

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://www.rfc-editor.org/info/rfc7519

Miller, D., Whitlock, J., Gardiner, M., Ralphson, M., Ratovsky, R., & Sarid, U. (2021). *OpenAPI specification* (Version 3.1.0). OpenAPI Initiative. https://spec.openapis.org/oas/v3.1.0.html

National Privacy Commission. (n.d.). *Data Privacy Act of 2012 (Republic Act No. 10173)*. Retrieved November 5, 2026, from https://www.privacy.gov.ph/data-privacy-act/

Nygard, M. (2011, November 15). *Documenting architecture decisions*. Cognitect. https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions

OWASP Foundation. (2023). *OWASP API security top 10 – 2023*. https://owasp.org/API-Security/editions/2023/en/0x11-t10/

Rescorla, E. (2018). *The Transport Layer Security (TLS) protocol version 1.3* (RFC 8446). Internet Engineering Task Force. https://www.rfc-editor.org/info/rfc8446

Richards, M., & Ford, N. (2020). *Fundamentals of software architecture: An engineering approach*. O'Reilly Media.

Wright, A., Andrews, H., Hutton, B., & Dennis, G. (2022). *JSON Schema: A media type for describing JSON documents* (Draft 2020-12). https://json-schema.org/draft/2020-12/

18. Appendices

AI use declaration

We declare that we reviewed, understood, and can explain and defend every part of this submission, including any part produced with AI assistance.

Signed: [signature of each member: Jessa Angela Sabueto, Alex Mendoza Jr., Joey Manlapas, Faith Red]

Tool	Used for	Part of the project	Member responsible
Claude	Drafting text, suggesting quality attribute scenarios, finding and checking references, drawing diagrams	Proposal parts 1–18	[name]

Meeting and consultation minutes

Date	Attendees	Topics discussed	Action items
[Date of Consultation 1]	[Names]	Scenario, parts 1–7 draft, repository	[Feedback applied in parts 1–7]
[Date of Consultation 2]	[Names]	Architecture and integration design	[Revisions requested]